

SIMATS ENGINEERING

CO4 - ASSESSMENT TOOL 2

Industry Problem Solving Task

COURSE NAME: Deep Learning

COURSE CODE: MLA0408

COURSE OUTCOME COVERED

CO 1: Students will be able to understand and apply probabilistic graphical models including Bayesian Networks, Markov Random Fields, Hidden Markov Models, and Conditional Random Fields. They will learn how to represent complex probabilistic relationships among variables and perform inference and learning in graphical models. Students will be able to use these models for reasoning under uncertainty and solving real-world decision-making problems. (BL-6)

Assessment Tool Weightage: 40%

QUESTIONS:

Total Marks:25

Q1. Transfer Learning

A healthcare startup aims to develop an AI-based system for detecting skin cancer from dermoscopic images. Since only a limited number of labeled images are available, design a transfer learning strategy using a pre-trained CNN model. Justify your choice of model, identify which layers should be frozen or fine-tuned, and explain how your design improves classification performance.

Q2. DenseNet and PixelNet

A smart city project requires an automated road-scene analysis system that accurately identifies roads, pedestrians, vehicles, and traffic signs at the pixel level. Design a deep learning architecture using DenseNet and PixelNet to solve this problem. Explain how the proposed architecture improves feature reuse, segmentation accuracy, and computational efficiency.

Q3. Bidirectional RNN and Encoder–Decoder

A multinational customer service company wants to build a multilingual chatbot capable of translating customer queries while preserving contextual meaning. Design an Encoder–Decoder sequence-to-sequence model using Bidirectional RNNs. Illustrate the architecture and justify how bidirectional processing improves translation accuracy.

Q4. BPTT and LSTM

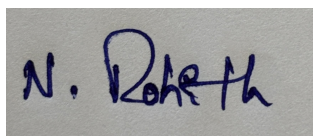
A financial analytics company is developing a model to forecast stock prices using historical market data. Create an LSTM-based prediction system and explain how Backpropagation Through Time (BPTT) is used to train the network. Justify why LSTM is more suitable than a traditional RNN for this application.

Q5. Integrated Deep Learning Solution

A media streaming company plans to automatically generate captions for uploaded videos to improve accessibility. Design an integrated deep learning solution that combines Transfer Learning for visual feature extraction with an LSTM-based Encoder–Decoder architecture for caption generation. Explain the role of each component and justify how the complete system produces accurate captions.

Code of Conduct:

I **NALIPI REDDY ROHITH/192425115**(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:**RUBRICS FOR CALCULATION:**

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis

Industry Problem Solving Task

Q1. Transfer Learning – Skin Cancer Detection

Problem

The healthcare startup has only a limited number of labeled dermoscopic images. Training a CNN from scratch would require a large dataset and may cause overfitting. Therefore, **transfer learning** is suitable.

Proposed Model: ResNet50

ResNet50 is selected because it is a powerful CNN with **residual/skip connections**, allowing it to learn deep features effectively.

Architecture

Dermoscopic Image



Resize + Normalization



Data Augmentation



Pre-trained ResNet50



Global Average Pooling



Dense Layer + ReLU



Dropout



Softmax



Benign / Malignant

Frozen and Fine-Tuned Layers

Initially, the **early convolutional layers** are frozen because they learn general features such as edges,

colors and textures.

The **last few convolutional/residual blocks** can be fine-tuned because they learn more specific features related to skin lesions.

The original classification layer is removed and replaced with a new layer for benign/malignant classification.

Data Augmentation

Suitable techniques include:

- Rotation
- Horizontal/vertical flipping
- Zooming
- Cropping
- Brightness adjustment

Why It Improves Performance

Transfer learning provides useful pre-trained features, reducing training time and the amount of labeled data required. Fine-tuning the deeper layers allows the model to adapt to dermoscopic patterns while freezing early layers reduces overfitting.

Conclusion: ResNet50 with selective fine-tuning provides an effective solution for skin-cancer classification when labeled medical data is limited.

code:

```
import numpy as np

import tensorflow as tf

from tensorflow.keras.applications import ResNet50

from tensorflow.keras.layers import Dense, GlobalAveragePooling2D, Dropout

from tensorflow.keras.models import Model

np.random.seed(42)

tf.random.set_seed(42)

X = np.random.rand(20, 224, 224, 3).astype("float32")
```

```
y = np.array([0,1,0,1,0,1,0,1,0,1,
              1,0,1,0,1,0,1,0,1,0])

base_model = ResNet50(
    weights="imagenet",
    include_top=False,
    input_shape=(224,224,3)
)

for layer in base_model.layers:
    layer.trainable = False

x = base_model.output
x = GlobalAveragePooling2D()(x)
x = Dense(64, activation="relu")(x)
x = Dropout(0.3)(x)
output = Dense(2, activation="softmax")(x)

model = Model(base_model.input, output)

model.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)
```

```

model.fit(X, y, epochs=2, batch_size=4, verbose=1)

test_image = np.random.rand(1,224,224,3).astype("float32")

prediction = model.predict(test_image, verbose=0)

classes = ["Benign", "Malignant"]

result = np.argmax(prediction)

print("\nSkin Cancer Prediction")

print("Result:", classes[result])

print("Confidence:",

      round(float(prediction[0][result]) * 100, 2), "%")

```

output:

```

... Epoch 1/2
5/5 ————— 10s 684ms/step - accuracy: 0.6000 - loss: 0.8325
Epoch 2/2
5/5 ————— 5s 1s/step - accuracy: 0.4000 - loss: 0.8580

Skin Cancer Prediction
Result: Benign
Confidence: 56.46 %

```

Q2. DenseNet and PixelNet – Road Scene Segmentation

Problem

The smart-city system must identify **roads, pedestrians, vehicles and traffic signs** at pixel level.

Therefore, semantic image segmentation is required.

Proposed Architecture

Road Image



DenseNet Backbone



Dense Feature Extraction



Multi-Level Feature Maps



Upsampling



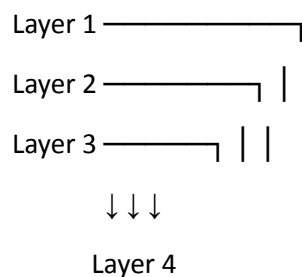
Pixel-wise Prediction



Semantic Segmentation Map

DenseNet Component

DenseNet connects each layer with subsequent layers so that previously learned features are reused.



This improves gradient flow and allows the network to reuse low-level and high-level features.

PixelNet Component

The extracted DenseNet features are passed to a PixelNet-style pixel-wise classifier. A 1×1 **convolution** can generate class predictions for individual pixels, followed by Softmax.

Each pixel is assigned a class such as:

- Road
- Pedestrian
- Vehicle

- Traffic sign
- Background

Advantages

Feature reuse: Dense connections reuse features from earlier layers.

Segmentation accuracy: Pixel-wise prediction provides detailed object boundaries.

Computational efficiency: Feature reuse reduces unnecessary repeated feature learning and improves information flow.

Conclusion: Combining DenseNet's feature extraction with PixelNet's pixel-level prediction provides an accurate and efficient road-scene segmentation system.

code:

```
import numpy as np

from tensorflow.keras.applications import DenseNet121
from tensorflow.keras.layers import Conv2D, UpSampling2D
from tensorflow.keras.models import Model

np.random.seed(42)

image = np.random.rand(
    1, 224, 224, 3
).astype("float32")

num_classes = 5

base_model = DenseNet121(
    weights="imagenet",
    include_top=False,
    input_shape=(224, 224, 3)
```

```
)

for layer in base_model.layers:

    layer.trainable = False

x = base_model.output

x = Conv2D(

    128,

    (3,3),

    padding="same",

    activation="relu"

)(x)

x = UpSampling2D(

    size=(8,8)

)(x)

output = Conv2D(

    num_classes,

    (1,1),

    padding="same",

    activation="softmax"

)(x)
```

```

model = Model(

    base_model.input,

    output

)

prediction = model.predict(

    image,

    verbose=0

)

segmentation = np.argmax(

    prediction[0],

    axis=-1

)

print("DenseNet + PixelNet Segmentation")

print("Input Shape:", image.shape)

print("Output Shape:", segmentation.shape)

print("\nPixel-wise segmentation completed.")

print("\nSample Pixel Classes:")

print(segmentation[100:105,100:105])

```

output:

```
... Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/densenet/29084464/29084464 3s 0us/step
DenseNet + PixelNet Segmentation
Input Shape: (1, 224, 224, 3)
Output Shape: (56, 56)

Pixel-wise segmentation completed.

Sample Pixel Classes:
[]
```

Q3. Bidirectional RNN and Encoder–Decoder

Problem

A multinational customer-service company requires a chatbot that can translate queries between languages while preserving their contextual meaning.

Proposed Architecture

Input Sentence



Word Embedding



Bidirectional RNN Encoder



Forward Backward



Context Vector



LSTM Decoder



Target Language



Translated Sentence

Encoder

The input sentence is converted into word embeddings. The Bidirectional RNN processes the

sentence in **two directions**:

- Forward: beginning → end
- Backward: end → beginning

The outputs from both directions are combined into a richer contextual representation.

Decoder

The decoder receives the context representation and generates the translated sentence one word at a time.

For example:

Input:

"Where is my order?"

Output:

"¿Dónde está mi pedido?"

Why Bidirectional Processing?

A normal RNN mainly uses information from previous words. A Bidirectional RNN can use information from both previous and following words.

This is useful when the meaning of a word depends on the complete sentence.

Advantages

- Better contextual understanding.
- Captures information from both directions.
- Helps handle ambiguous words.
- Improves translation quality.
- Suitable for multilingual applications.

Conclusion: A Bidirectional RNN encoder combined with an LSTM decoder provides a strong architecture for context-aware multilingual translation.

code:

```
import numpy as np

import tensorflow as tf

from tensorflow.keras.models import Model

from tensorflow.keras.layers import (
```

```
Input,

Embedding,

Bidirectional,

LSTM,

Dense,

RepeatVector
)

np.random.seed(42)

tf.random.set_seed(42)

vocab_size = 20

sequence_length = 5

X = np.array([

    [1,2,3,0,0],

    [1,4,3,0,0],

    [1,5,6,3,0],

    [1,2,7,3,0]

])

y = np.array([

    [1,8,9,3,0],

    [1,10,9,3,0],

    [1,11,12,9,3],
```

```
[1,8,13,9,0]

])

encoder_input = Input(

    shape=(sequence_length,)

)

embedding = Embedding(

    vocab_size,

    32

)(encoder_input)

encoder_output = Bidirectional(

    LSTM(32)

)(embedding)

context = RepeatVector(

    sequence_length

)(encoder_output)

decoder = LSTM(

    64,

    return_sequences=True

)(context)
```

```
output = Dense(
    vocab_size,
    activation="softmax"
)(decoder)

model = Model(
    encoder_input,
    output
)

model.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)

target = y.reshape(
    y.shape[0],
    y.shape[1],
    1
)

model.fit(
    X,
    target,
```

```
        epochs=20,

        batch_size=2,

        verbose=0
    )

test_sentence = np.array([

    [1,2,3,0,0]

])

prediction = model.predict(

    test_sentence,

    verbose=0
)

translation = np.argmax(

    prediction,

    axis=-1
)

print("Bidirectional RNN Encoder-Decoder")

print("Translation Prediction:")

print(translation)

print("\nPrediction Shape:")

print(prediction.shape)
```

output:

```
... WARNING:tensorflow:5 out of the last 5 calls to <function TensorFlowTrai
Bidirectional RNN Encoder-Decoder
Translation Prediction:
[[9 9 9 0 0]]

Prediction Shape:
(1, 5, 20)
```

Q4. BPTT and LSTM – Stock Price Prediction

Problem

Stock prices are time-series data. Previous market conditions can influence future prices, so the model must learn relationships across multiple time steps.

Proposed Architecture

Historical Market Data



Data Cleaning



Normalization



Time-Series Sequences



LSTM Layer



Dense Layer



Predicted Stock Price

Input features may include:

- Opening price
- Closing price
- Highest price

- Lowest price
- Trading volume

LSTM Structure

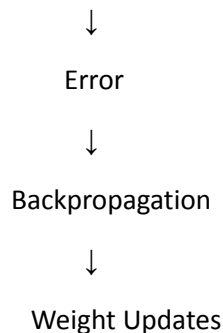
LSTM contains:

- **Forget gate** – removes unnecessary information.
- **Input gate** – stores important new information.
- **Output gate** – determines the information passed forward.
- **Cell state** – carries useful information across time.

BPTT Training

During training, the LSTM is unrolled over the sequence:

Day 1 → LSTM → Day 2 → LSTM → Day 3 → LSTM → Prediction



The prediction error is propagated backward through the previous time steps. Gradients are calculated and the network weights are updated using an optimizer such as Adam.

This process is called **Backpropagation Through Time (BPTT)**.

Why LSTM Instead of RNN?

Traditional RNNs can suffer from **vanishing gradients**, making it difficult to learn long-term dependencies.

LSTM uses gates and a cell state to preserve important information for longer periods.

Evaluation

The model can be evaluated using:

- MSE
- RMSE
- MAE
- MAPE

Conclusion: LSTM is more suitable than a standard RNN because it can preserve long-term

information and handle sequential financial data more effectively.

code:

```
import numpy as np

import tensorflow as tf

from tensorflow.keras.models import Sequential

from tensorflow.keras.layers import LSTM, Dense


np.random.seed(42)

tf.random.set_seed(42)


prices = np.array([

    100,102,104,103,106,

    108,110,109,112,115,

    117,116,120,122,125,

    124,128,130,133,135

], dtype=float)


minimum = prices.min()

maximum = prices.max()


scaled = (

    prices - minimum

) / (

    maximum - minimum

)
```

```
X = []

y = []

sequence_length = 3

for i in range(
    len(scaled) - sequence_length
):
    X.append(
        scaled[i:i+sequence_length]
    )
    y.append(
        scaled[i+sequence_length]
    )

X = np.array(X)
y = np.array(y)

X = X.reshape(
    X.shape[0],
    X.shape[1],
    1
)
```

```
model = Sequential([

    LSTM(32, input_shape=(3,1)),

    Dense(1)

])

model.compile(

    optimizer="adam",

    loss="mse"

)

model.fit(

    X,

    y,

    epochs=30,

    batch_size=4,

    verbose=0

)

print("LSTM Training Completed.")

print("BPTT is used during LSTM training.")

last_three = scaled[-3:]

test_input = last_three.reshape(

    1,3,1
```

```

)

prediction = model.predict(

    test_input,

    verbose=0

)

predicted_price = (

    prediction[0][0]

    * (maximum-minimum)

    + minimum

)

print(

    "Predicted Next Closing Price:",

    round(float(predicted_price),2)

)

```

output:

```

... /usr/local/lib/python3.13/dist-packages/keras/src/layers/rnn
    super().__init__(**kwargs)
WARNING:tensorflow:6 out of the last 6 calls to <function Te
LSTM Training Completed.
BPTT is used during LSTM training.
Predicted Next Closing Price: 136.14

```

Q5. Integrated Deep Learning Solution – Video Caption Generation

Problem

The media company wants to automatically generate captions for videos. The system must understand both **visual content** and **temporal relationships** between video frames.

Proposed Architecture

Video



Frame Extraction



Pre-trained CNN



Visual Feature Extraction



LSTM Encoder



Context Vector



LSTM Decoder



Word Prediction



Generated Caption

1. Transfer Learning

A pre-trained CNN such as **ResNet50** or **MobileNetV2** is used to extract visual features from individual frames.

The CNN is initially frozen because it already contains useful features such as:

- Objects
- Shapes

- Colors
- Textures
- People
- Backgrounds

Selected deeper layers can later be fine-tuned if a suitable video dataset is available.

2. Frame Processing

The video is divided into a sequence of frames:

Frame 1 → Frame 2 → Frame 3 → Frame 4 → Frame 5

Each frame is passed through the CNN to obtain a feature vector.

3. LSTM Encoder

The sequence of visual feature vectors is given to an LSTM encoder.

The encoder learns **temporal relationships** between frames.

For example:

Person → picks up ball → throws ball

The LSTM can understand that these frames represent one continuous activity.

4. LSTM Decoder

The decoder uses the encoded information to generate a caption sequentially.

<START>

↓

A

↓

boy

↓

is

↓

playing

↓

football

↓

<END>

The next word is predicted based on the previous generated words and the video context.

5. Role of Each Component

Component	Role
CNN	Extracts visual features
Transfer Learning	Reuses pre-trained visual knowledge
LSTM Encoder	Learns temporal information
Context Vector	Represents important video information
LSTM Decoder	Generates caption sequentially
Softmax	Selects the most probable next word

6. Training

The CNN extracts features from training videos. These features are provided to the LSTM encoder-decoder along with their corresponding captions.

The model is trained using **categorical cross-entropy loss** and an optimizer such as Adam.

During training, the decoder learns to predict the next word from the previous words and video representation.

7. Example

If the video shows a boy playing football, the model may generate:

"A boy is playing football."

If the video shows a person walking:

"A man is walking on the road."

8. Evaluation

Suitable metrics include:

- BLEU
- METEOR
- ROUGE
- CIDEr

These compare generated captions with reference captions.

9. Advantages

- Transfer learning reduces the need for a huge image dataset.
- CNN extracts meaningful visual information.
- LSTM captures temporal relationships.
- Encoder-decoder architecture generates captions word by word.
- The system can improve accessibility for visually impaired users.

10. Conclusion

The integrated CNN–LSTM architecture combines **visual feature extraction and temporal sequence learning**. Transfer learning understands what appears in the video, while the LSTM learns how the visual content changes over time and converts it into meaningful natural-language captions.

code:

```
import numpy as np

import tensorflow as tf

from tensorflow.keras.applications import MobileNetV2
from tensorflow.keras.layers import Input, LSTM, Dense
from tensorflow.keras.models import Model

np.random.seed(42)

tf.random.set_seed(42)

cnn = MobileNetV2(
    weights="imagenet",
```

```
        include_top=False,

        pooling="avg",

        input_shape=(224,224,3)
    )

for layer in cnn.layers:

    layer.trainable = False

# Simulated video frames
frames = np.random.rand(

    5,224,224,3

).astype("float32")

# CNN feature extraction
features = cnn.predict(

    frames,

    verbose=0

)

features = features.reshape(

    1,5,1280

)

# LSTM Encoder
encoder_input = Input(
```

```

        shape=(5,1280)

    )

encoder = LSTM(

    128,

    return_state=True

)

encoder_output, state_h, state_c = encoder(

    encoder_input

)

# LSTM Decoder

decoder_input = Input(

    shape=(5,9)

)

decoder = LSTM(

    128,

    return_sequences=True

)

decoder_output = decoder(

    decoder_input,

    initial_state=[state_h,state_c]

```

```
)

output = Dense(

    9,

    activation="softmax"

)(decoder_output)

model = Model(

    [encoder_input, decoder_input],

    output

)

model.compile(

    optimizer="adam",

    loss="categorical_crossentropy"

)

decoder_data = np.random.rand(

    1, 5, 9

).astype("float32")

prediction = model.predict(

    [features, decoder_data],

    verbose=0

)
```

```
words = [

    "a",

    "boy",

    "is",

    "playing",

    "football",

    "girl",

    "running",

    "man",

    "walking"

]

predicted_words = np.argmax(

    prediction[0],

    axis=1

)

caption = [

    words[i]

    for i in predicted_words

]

print("Video Caption Generation")

print("Feature Shape:", features.shape)
```

```
print("Prediction Shape:", prediction.shape)

print("\nGenerated Caption:")

print(" ".join(caption))
```

output:

```
... Downloading data from https://storage.googleapis.com/tensorflow/tfjs/tfjs-models/9406464/9406464 2s 0us/step
Video Caption Generation
Feature Shape: (1, 5, 1280)
Prediction Shape: (1, 5, 9)

Generated Caption:
man man man man man
```